My plan for automated alignment search fell apart. Here is what it found instead.

Surya V

October 30, 2026

TL;DR. I built an automated search that looks for the places in a network where a causal variable lives. I hoped it would let me write down a model of a task and get the mechanism back. In practice it returns the place where the variable is already visible: the token that holds the answer, or the final position where the answer is written out. I tested this on GPT-2 small (IOI and sequence completion) after splitting each task into groups with Data-driven Circuit Discovery. The search scored very well on IOI, and it did so by picking the indirect object's name token. On sequence completion it only worked when allowed to patch the final position. My conclusion is that if the causal variable is explicit in the model (it is an input or output token), the search simply reads it off, and the search can only tell you something new when the high-level variable is indirect *and* the assumption you made about that variable is true.

1 Where I started

I created this automated alignment search, and my hopes for it were large. The idea was that interpretability could become something you run rather than something you do by hand. You write down a high-level causal model of a task, for example "the ones digits produce a carry, the carry and the tens digits produce the answer", point the search at the network, and it tells you which activations play the role of each variable. Do this for enough tasks and you get a map of what the network computes, without hunting for circuits head by head.

That hope did not survive contact with the results. The first two experiments gave sites that "worked" but did not look like computation, and when I redid the work on two well-studied tasks the same thing happened more cleanly. This post is an account of that, and of what I now think the search can and cannot do.

2 The method

A *site* is a place in the residual stream, given by a layer ℓ and a token position p. Take a base prompt B and a source prompt X, and let h_i^B and h_i^X be the activations at site i. For a binary mask m over sites, the *interchange intervention* replaces the base activation by the source activation wherever $m_i = 1$:

$$h_i^{\text{int}} = (1 - m_i)\, h_i^B + m_i\, h_i^X = h_i^B + m_i\, (h_i^X - h_i^B), \tag{1}$$

and the forward pass continues from there. The high-level model makes a prediction for what should happen: $y^\star(B, X)$ is the answer that model gives when the aligned variables are set to their values on X. The score is the interchange intervention accuracy,

$$\mathrm{IIA}(m) = \frac{1}{n}\sum_{j=1}^{n} \mathbf{1}\big[\hat{y}(B_j \leftarrow X_j;\, m) = y_j^\star\big], \tag{2}$$

the fraction of pairs where the patched network gives the predicted answer.

Learning the mask with a straight-through estimator (STE). The mask has to be 0 or 1, because a half-patched activation is not an intervention anyone can describe. But a hard threshold has zero gradient almost everywhere, so I cannot train it directly. The straight-through estimator fixes this: each site has a real-valued logit α_i, the *forward* pass uses the hard mask $\mathbf{1}[\alpha_i > 0]$, and the *backward* pass pretends the mask was the smooth sigmoid $\sigma(K\alpha_i)$:

$$m_i^{\text{used}} \;=\; \mathbf{1}[\alpha_i > 0] \;-\; \mathrm{sg}\big[\sigma(K\alpha_i)\big] \;+\; \sigma(K\alpha_i), \qquad \frac{\partial m_i^{\text{used}}}{\partial \alpha_i} = K\,\sigma(K\alpha_i)\big(1-\sigma(K\alpha_i)\big), \tag{3}$$

where $\mathrm{sg}[\cdot]$ stops the gradient. The value of m^{used} is exactly the hard 0/1 mask, so every loss I compute comes from a real intervention, while the gradient still tells each logit which way to move. The loss rewards the counterfactual answer and charges a price for every site selected:

$$\mathcal{L}(\alpha) = \mathbb{E}_{(B,X)}\Big[-\log p\big(y^\star \,\big|\, B \leftarrow X;\, m^{\text{used}}(\alpha)\big)\Big] + \lambda \sum_i \sigma(K\alpha_i). \tag{4}$$

The selected set is $S = \{i : \alpha_i > 0\}$ after Adam. I used $\lambda = 0.1$, $K = 4$, 350 full-batch epochs and learning rate 5×10^{-3}. For the earlier experiments I also used a variant that patches only a low-dimensional subspace at each selected site (in the style of DAS, Geiger et al., 2024): $h_i^{\mathrm{DAS}} = h_i^B + m_i\, Q_i Q_i^\top (h_i^X - h_i^B)$, with Q_i an orthonormal basis.

3 Two earlier attempts

Ones-column carry in Qwen3-1.7B. Prompts look like $a + b =$, digits are separate tokens, and the high-level variable is the carry out of the ones column, $C = \mathbf{1}[a_{\mathrm{o}} + b_{\mathrm{o}} \geq 10]$. Pairs changed only the ones digits. The best subspace search (rank 8 at layer 6, position 7) reached 81% on the training pairs but 51% held out. Reversing the intervention (carry to no carry) gave 49%. Control pairs where the carry stays the same should have been left unchanged, and instead failed (78–90% correct, where 100% is expected). In unrestricted searches the optimiser picked the final token at layers 24 and 27, where the answer is already there, so I banned late layers by hand.

Graph successor in a small Transformer. A 3-layer model trained from scratch on shuffled paths; the target is the next node after the query node. The search selected the query-node position at layer 2 with 100% IIA, but a full patch at *any* of three layers of the query node also gave 99–100%, and the learned one-direction subspaces gave only 56%. Random masks of the same size already scored 60–93%.

In both cases I had decided the variable, the sites and the pairs in advance and the search handed my assumptions back to me. Instead of fixing these one at a time, I wanted a setting with a known mechanism, so I moved to GPT-2 small.

4 Splitting the tasks into groups with Data-driven Circuit Discovery

Data-driven Circuit Discovery (DCD; Rai, Geva and Yao, 2026, arXiv:2605.09129) argues that a single circuit for a whole task can hide the fact that different examples are handled in different ways. Its remedy is to compute an attribution for every example and then group examples whose attributions look alike. I followed its recipe, then ran my alignment search inside each group.

1. **Attribution per example.** For each of 800 examples (600 training, 200 validation) I computed edge attributions with EAP-IG (Hanna et al.), 5 integration steps, over the 32,491 edges of GPT-2 small. The metric is the logit margin on the clean prompt. For IOI,

$$m_{\mathrm{IOI}}(x) = \mathrm{logit}(\mathrm{IO} \mid x) - \mathrm{logit}(\mathrm{S} \mid x), \tag{5}$$

   and for sequence completion, with target E and the other five motif tokens as foils,

$$m_{\mathrm{seq}}(x) = \mathrm{logit}(E \mid x) - \tfrac{1}{5} \sum_{w \in \{A,B,C,D,F\}} \mathrm{logit}(w \mid x). \tag{6}$$

   The corrupted prompt used for attribution replaces the second occurrence of the subject in IOI, and replaces two of the motif tokens everywhere they occur in sequence completion.

2. **Binarise.** For each example keep the smallest set of edges that carries 99% of the total absolute attribution, $b_e(x) = \mathbf{1}[e \in T_{0.99}(x)]$.

3. **Reduce and cluster.** PCA to 20 dimensions (these keep 12.5% of the variance for IOI and 10.0% for sequence completion), then K-means. PCA and K-means were fitted on the training examples only; validation examples were assigned to the fitted clusters.

4. **Choose the number of clusters** with the gap statistic and the one-standard-error rule (Tibshirani, Walther and Hastie, 2001), over $k \in \{2, \ldots, 10\}$:

$$\mathrm{Gap}(k) = \mathbb{E}^*[\log W_k] - \log W_k, \qquad \hat{K} = \min\{k : \ \mathrm{Gap}(k) \geq \mathrm{Gap}(k{+}1) - s_{k+1}\}, \tag{7}$$

   where W_k is the within-cluster spread and $\mathbb{E}^*$ is an average over 20 uniform reference data sets. This chose $\hat{K} = 8$ for IOI and $\hat{K} = 6$ for sequence completion.

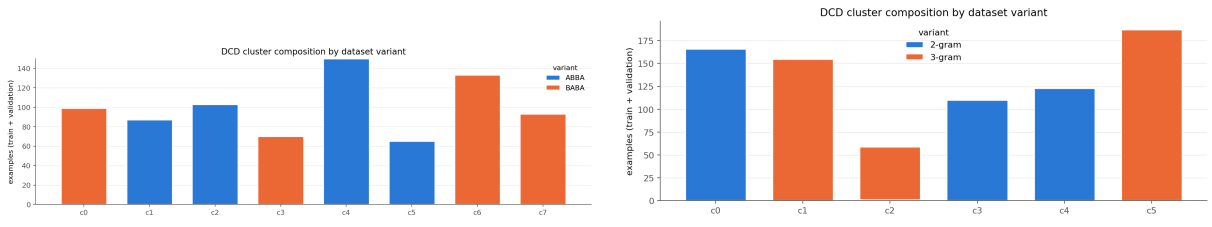


Figure 1: Composition of the DCD clusters by dataset variant. Left, IOI: every cluster contains one template (ABBA or BABA). Right, sequence completion: every cluster contains one variant (2-gram or 3-gram), apart from one validation example.

Every cluster turned out to be one template or variant. For IOI that is informative: the two templates differ in which of the two first-clause names is repeated, which the IOI literature (Wang et al., 2023) says matters. For sequence completion the two variants differ only in the corrupted prompt used for attribution, so I would not read the six clusters as six different mechanisms. The gap statistic curve for IOI keeps rising up to $k = 10$, so the exact count is not sharply determined.

Pairs for the search. Inside each cluster (and so inside one template or variant), I paired each base prompt with a source prompt that has a different answer, and kept only pairs where the model answers the base correctly. The target is the source's answer. Without any patching the success rate is zero. Training, validation and test pairs are built separately, and the test split was used once at the end.

Candidate sites. Layers 3 to 8. I only allowed token positions in the span where base and source tokens actually differ, and I excluded the final position. The reasons: positions before the first difference have identical activations in both prompts, so patching them does nothing; and patching the final position just overwrites the model's prediction with the source's. That leaves 54 sites for IOI (positions 2–10) and 90 for sequence completion (positions 4–18).

5 IOI: the search finds the indirect object's name

An IOI prompt is "When Mary and John went to the store, John gave a drink to" and the answer is Mary. There are two templates, ABBA (indirect object first) and BABA (indirect object second). Validation IIA was between 0.90 and 1.00 in every cluster, 174 of 182 pairs overall, against 44% for random masks of the same size. On the test split it was 175 of 184 (95%).

Cluster	Template	Nodes	Train	Val	Test	Random (val)	All sites (val)
0	BABA	8	1.00	0.91	1.00	0.23	0.91
1	ABBA	7	0.96	0.93	0.92	0.45	1.00
2	ABBA	9	1.00	1.00	1.00	0.51	1.00
3	BABA	8	0.98	1.00	1.00	0.47	1.00
4	ABBA	6	0.94	0.95	0.90	0.39	0.97
5	ABBA	6	0.93	1.00	1.00	0.64	1.00
6	BABA	8	0.92	0.90	0.92	0.46	0.97
7	BABA	8	0.93	1.00	0.96	0.48	1.00

Table 1: IOI, per cluster. IIA of the learned mask on training, validation and test pairs, the mean of random masks with the same number of nodes, and patching every candidate site. Patching with no mask gives 0 everywhere.

Of the 60 selected nodes, 48 are the six nodes (layers 3 to 8) on the token that holds the indirect object's name. The remaining 12 are on the repeated subject (position 10), the word "and" (position 3) and, in one cluster, the other name.

Does the search need all of them? Leave-one-out. To see which selected nodes matter, I removed one node at a time from each learned mask and re-measured validation IIA (figure below; the dashed line is the full mask).

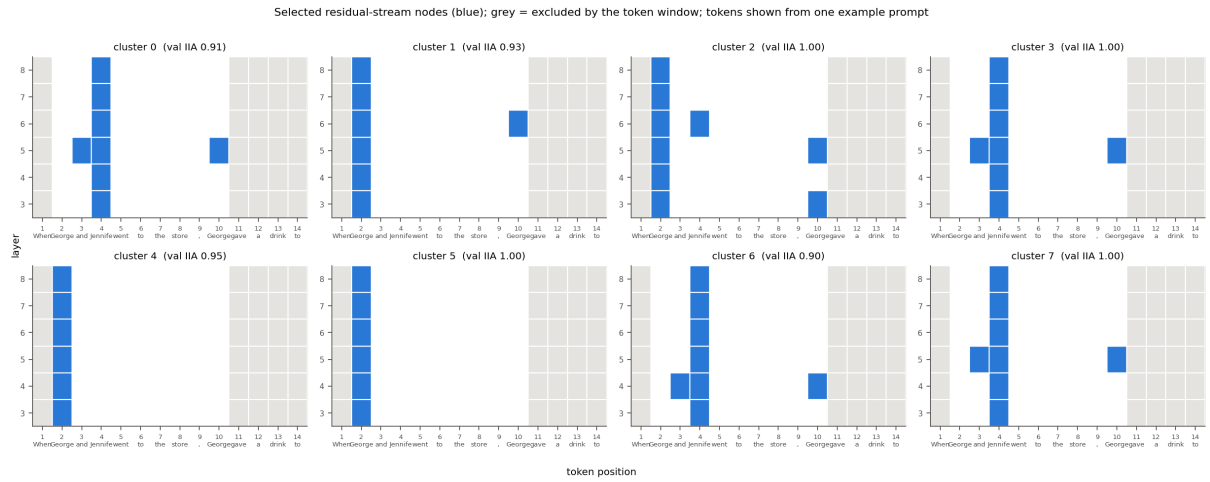


Figure 2: IOI: selected nodes (blue) per cluster. Grey is outside the token window. The search lands on the indirect object's name in every cluster (position 2 for ABBA, position 4 for BABA).

Removing any single node from the indirect-object column changes almost nothing (the bars sit on the dashed line), because the other five nodes in that column still carry the swapped name. The few nodes that do hurt are outside the column, on the "and" and repeated-subject positions: in cluster 6, removing $(4, 3)$ or $(4, 10)$ drops IIA from 0.90 to about 0.3, and in cluster 0 removing $(5, 10)$ drops it from 0.91 to 0.68. So the column is highly redundant, and a handful of extra nodes carry the part that the name alone does not supply. I did not test why these particular nodes matter.

One token is enough. If I patch layers 3 to 8 at a single position and nothing else, the name tokens alone reach IIA close to 1, and every other position gives 0 (left panel below). Swapping the name token swaps "the indirect object is Mary" for "the indirect object is Alice", and the later layers carry this through to the answer. That shows the name is sufficient. It says nothing about the heads that decide which of the two names is the indirect object, and that decision is the interesting part of the circuit.

Letting the search use all 14 positions, or all but the final one, changed nothing: the same nodes were selected and IIA was the same. The IOI search never wanted the final position at all. The masks mostly transfer between clusters of the same template and mostly not across templates (cluster 2's mask is an exception that transfers broadly). Since I paired prompts only within a template, the variable that differs between the templates, which name is repeated, never varied in the data, so the search could not have found it.

6 Sequence completion: the final position, or nothing

Each prompt is a six-token motif repeated twice and then cut after four tokens; the model should give the fifth, $ABCDEF\ ABCDEF\ ABCD \rightarrow E$. I used a 2-gram variant (motif $ABCDEF$) and a 3-gram variant (motif $ABCXEF$), 500 prompts each. GPT-2 small gets 99.7% of these right.

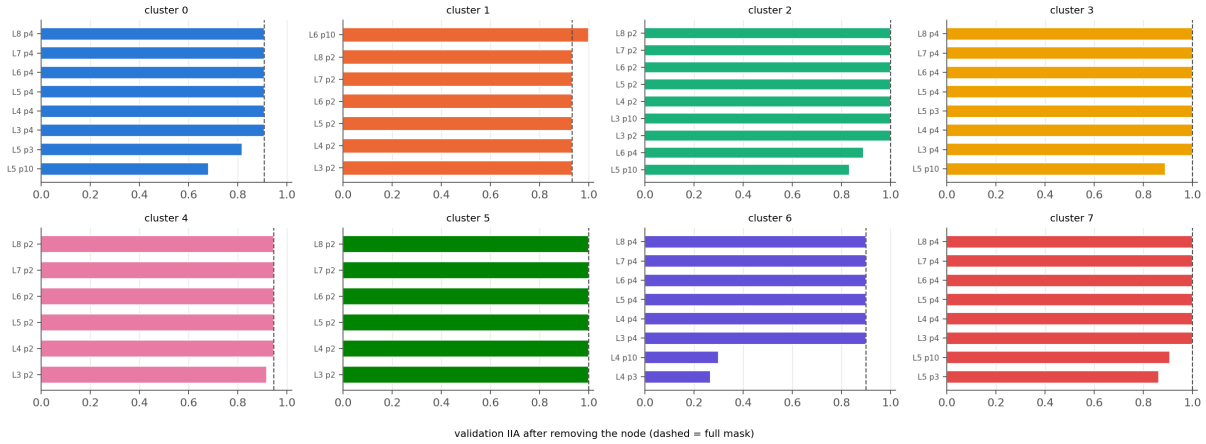


Figure 3: IOI leave-one-out: validation IIA after removing each selected node, for each cluster. Dashed line: the full mask.

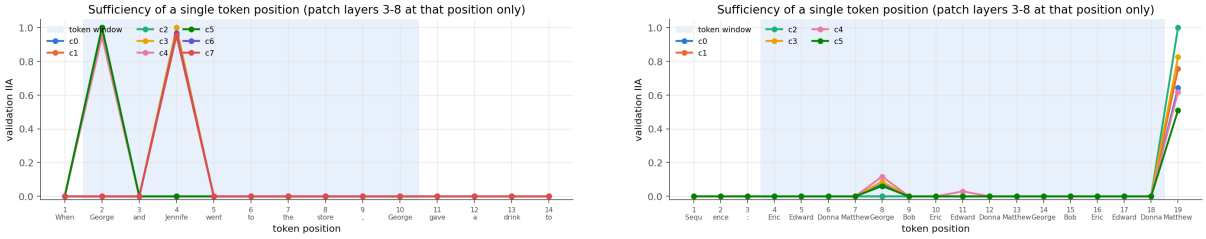


Figure 4: Validation IIA from patching layers 3–8 at one token position only. Left, IOI: only the two name tokens work. Right, sequence completion: only the final position works.

With the final position excluded, the search did poorly. Validation IIA was between 0.00 and 0.71 depending on the cluster, 70 of 198 pairs overall (35%), against 6% for random masks. On the test split it was 63 of 199 (32%). Patching every candidate site gave only 0–0.22, so there is no easy cut available in this window at all.

Cluster	Variant	Nodes	Train	Val	Test	Random (val)	All sites (val)
0	2-gram	8	0.72	0.60	0.55	0.04	0.19
1	3-gram	7	0.31	0.38	0.27	0.07	0.19
2	3-gram	8	0.04	0.00	0.00	0.02	0.00
3	2-gram	11	0.09	0.09	0.19	0.07	0.04
4	2-gram	6	0.60	0.71	0.77	0.06	0.21
5	3-gram	7	0.13	0.10	0.06	0.07	0.22

Table 2: Sequence completion, per cluster, final position excluded (columns as in the IOI table).

When I let the search use the final position again, validation IIA was 1.00 in all six clusters, and about a quarter of the selected nodes were at the final position. This agrees with my first, unrestricted run, where every selected node was on an occurrence of the answer token or at the final position (layers 7–9). Rerunning the restricted search gave values of 0.03–0.21 per cluster, so the restricted result is also not stable from run to run.

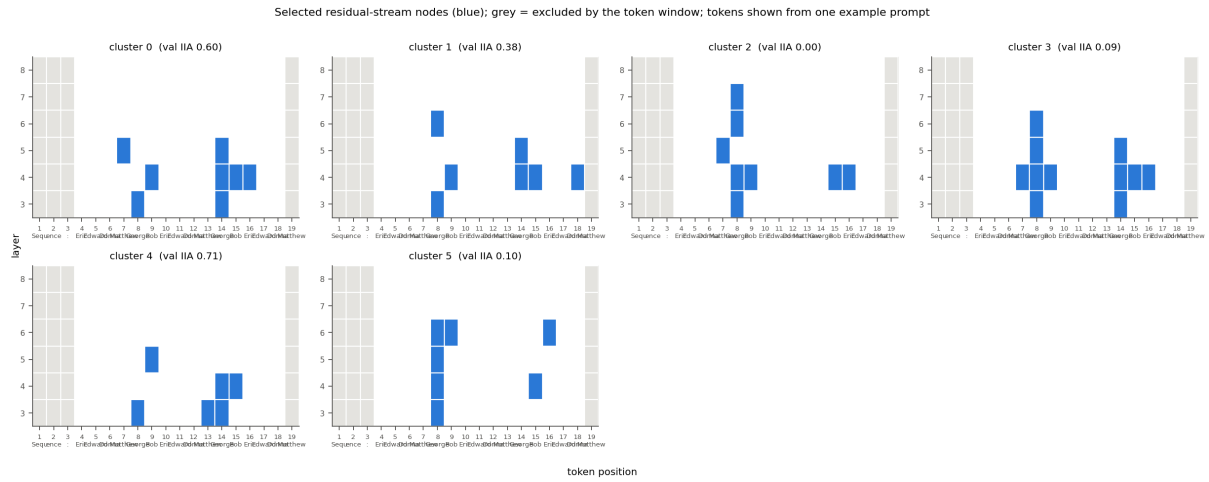


Figure 5: Sequence completion with the final position excluded. The nodes are spread over the first and second copies of the motif and there is no single dominant site.

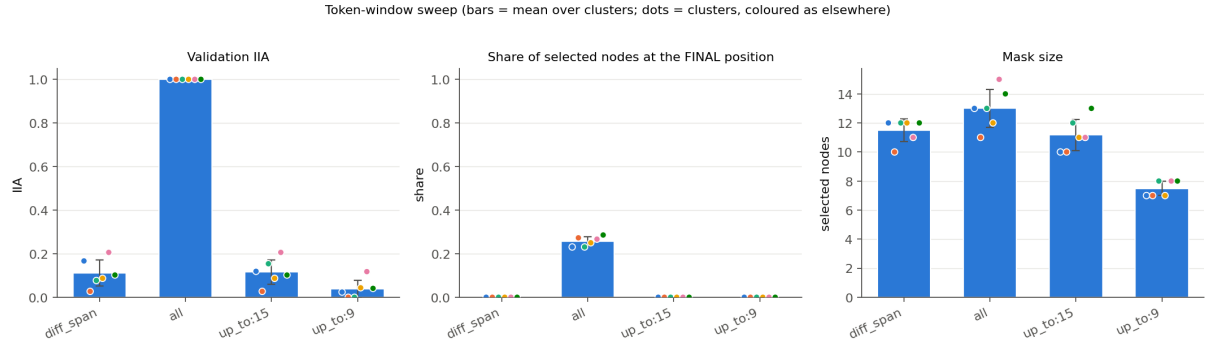


Figure 6: Sequence completion: the same search under different token windows. "all" includes the final position; the other windows do not. Validation IIA goes from about 1 to about 0.1, and the share of nodes at the final position (middle) shows where the 1.0 comes from.

The single-position test (right panel of the earlier figure) shows the same thing from the other side: patching the final position alone gives IIA between 0.5 and 1.0, while the first copy of the answer token E (position 8) gives close to 0. The answer token sits in the prompt, but swapping it does not swap the answer, because the model finds it through the repeated prefix. The information is moved to the last position, and that is the only narrow place left.

Leave-one-out looks different here than for IOI. Most nodes can be removed with no change, or even a small gain (removing $(6, 8)$ in cluster 1 raises IIA from 0.38 to 0.59). The one node that is clearly needed in the two best clusters is $(3, 8)$, the first copy of the answer token at layer 3; removing it takes IIA to 0. So the little that works rests on a single early node, and the mask has no larger structure.

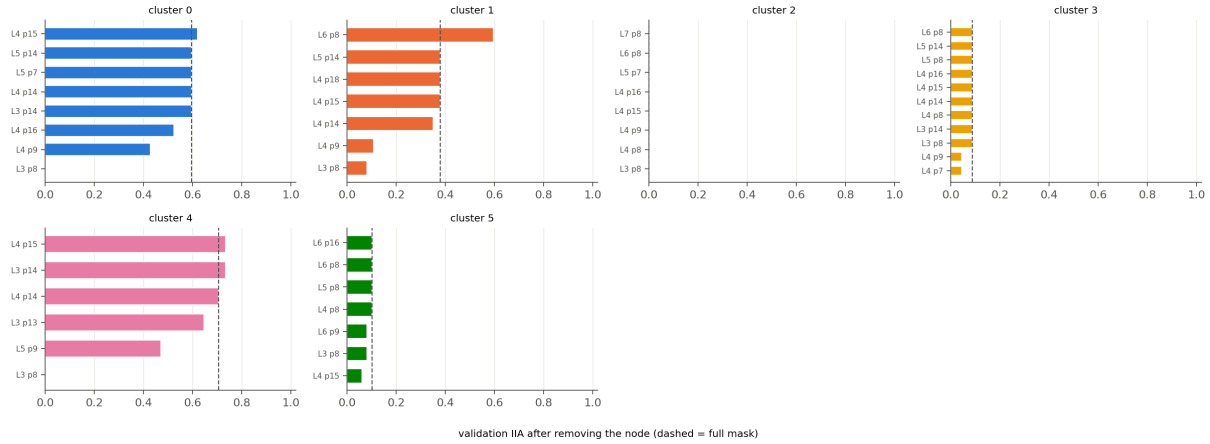


Figure 7: Sequence completion leave-one-out (final position excluded). Removing most nodes changes little, and in a few cases raises IIA. The node $(3, 8)$, the first copy of E at layer 3, drops IIA to 0 in clusters 0 and 4.

7 Why the search behaves this way

Say the base and source prompts differ on some set of input tokens $D(B, X)$. Call a set of sites S a *cut* if every path through the network from $D(B, X)$ to the output passes through S.

Proposition 1. If S is a cut, then $\hat{y}(B \leftarrow X; S) = \hat{y}(X)$. So when the target is the source's answer, $\mathrm{IIA}(S)$ equals the model's accuracy on the source prompts.

Proof sketch. Go through the nodes in the order the network computes them. A node that is not downstream of the difference has the same value in both runs. A node in S takes the source's value. Every path from the difference to the output crosses S, so the output depends only on values that agree with the source run.

An answer-level target is therefore satisfied by *every* cut, and the penalty $\lambda \sum_i \sigma(K\alpha_i)$ picks a small one. In a Transformer with causal attention the small cuts are at the two ends. At the input end, what enters at token p travels onward only through the stream at p, so the stream at p from some layer on is a narrow cut for what entered there (the indirect object's name). At the output end, the final position in late layers is a narrow cut for what has already been moved there. In the middle, the information is spread over many heads and positions, so a cut through it is wide and the penalty avoids it. That is the pattern in the IOI and sequence figures. It also explains why patching every site gives IIA near 1 whenever the final position is included: it is an identity, not evidence.

8 Conclusion

- **If the causal variable is explicit in the model, the search reads it off as it is.** When the variable is a token that appears in the input, or the answer token at the output, the cheapest way to change the behaviour is to swap that token's own representation. The search finds that representation and reports it with very high IIA. This is true but not

informative: it tells you where the variable is written down, not where it is computed. IOI's indirect object name and the final position in sequence completion are exactly this.

- **So the search can only teach you something when the high-level variable is indirect**, meaning it is not simply an input or the output, and the interchange target is a counterfactual answer that is neither the base's answer nor the source's. For IOI, an example is a variable for the position of the repeated name,

$$P = \begin{cases} 1 & N_3 = N_1 \\ 2 & N_3 = N_2 \end{cases}, \qquad Q = 3 - P, \qquad \text{Out} = N_Q.$$

  With an ABBA base and a BABA source, setting P to its source value makes the model say the *subject*, which is neither answer. A site that only carries the indirect object's name cannot produce that.

- **Even then it only works if my assumption about the variable is true.** The search will find sites for whatever variable I supply, and a good IIA only says that this variable, with these pairs, is sufficient. The carry experiment shows the other side: if the network does not compute the carry the way I wrote it down, the held-out, reversed and same-carry tests fail, and nothing tells the search that my model was wrong. Pairs also matter. If pairs differ only in the parents of a variable, IIA cannot separate the variable from its parents, so I need pairs where the parents differ and the variable does not.

So the vision I started with, where a high-level model goes in and a mechanism comes out, does not hold up as a fully automatic procedure. What survives is a good way to *test* a mechanism I already have a reason to believe in: write the model with its intermediate variables, build pairs in which each variable changes on its own, search over all layers and positions, and check that the sites found also pass reversed, same-value and multi-variable interventions.

References.

Rai, Geva and Yao. *Data-driven circuit discovery for interpretability of language models.* arXiv:2605.09129, 2026.

Geiger, Wu, Potts, Icard and Goodman. *Finding alignments between interpretable causal variables and distributed neural representations.* CLeaR, 2024.

Hanna et al. EAP-IG: edge attribution patching with integrated gradients (software), https://github.com/hannamw/EAP-IG.

Tibshirani, Walther and Hastie. *Estimating the number of clusters in a data set via the gap statistic.* JRSS B, 2001.

Wang, Variengien, Conmy, Shlegeris and Steinhardt. *Interpretability in the wild: a circuit for indirect object identification in GPT-2 small.* ICLR, 2023.